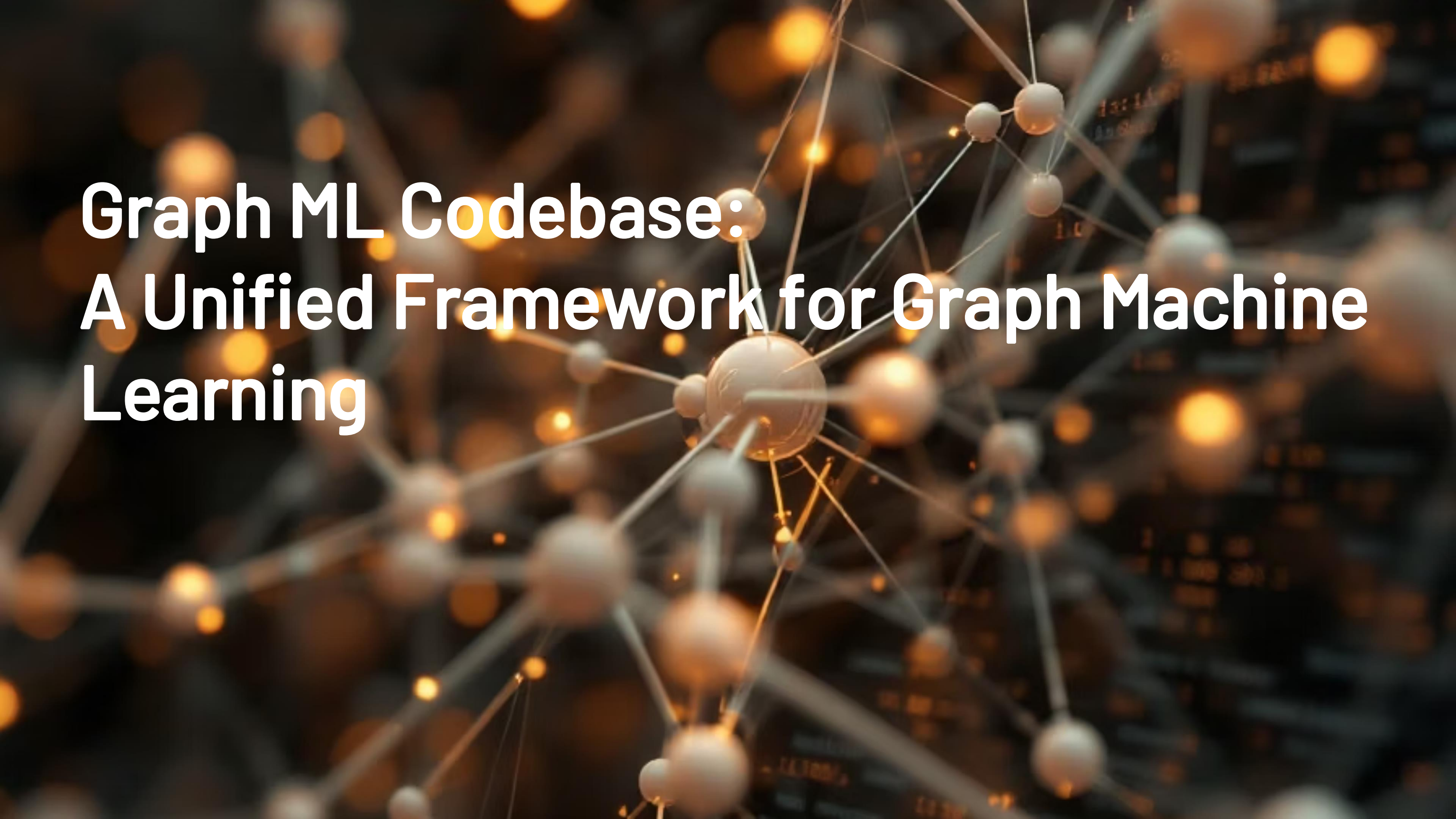




Graph ML Codebase: A Unified Framework for Graph Machine Learning

Introduction:

Build a unified Graph ML framework for diverse graph-based tasks.

An extensible codebase designed for developing graph based ML models

Structured implementation of Graph Neural Networks (GNNs)

Includes 12 tasks integrated with multiple datasets



Project Goals:

1

User Friendly for

Design and implement an extensible Graph ML codebase that supports a variety of graph learning tasks at the node, edge, and graph levels.

2

Tasks

The framework currently includes 12 tasks:

- **Node-level:** Classification, Regression, Clustering, Embedding
- **Edge-level:** Classification, Regression, Link Prediction
- **Graph-level:** Classification, Regression, Matching, Generation, Reconstruction

3

Clean and efficient code

- One line bash command to execute the program
- All python files in a dedicated folder
- All training and testing data in a dedicated folder

System Design:

1

Dataset Loading

Supports multiple datasets such as Cora, MUTAG, MovieLens etc. Loader scripts handle format parsing and graph structure extraction.

3

Model Selection

Supports a variety of models such as GCN, GAT, GraphSAGE, Siamese GCN, GMN, GAE, GRAE, GraphVAE and GraphRNN.

5

Training and Evaluation

Conducts forward propagation, loss computation, back propagation, and evaluation using metrics such as accuracy, RMSE, AUC, and F1-score.

2

Data Preprocessing

Includes node/edge feature extraction, adjacency matrix generation, normalization, and train-test splitting.

4

Task Modules

Implements tasks across three levels: Node (Classification, Regression, Embedding, Clustering), Edge (Classification, Regression, Link Prediction), and Graph (Classification, Regression, Matching, Generation, Reconstruction).

6

Model Saving

Saves trained models for a specific dataset and task in a pt file format for future use.

Software Components and Implementation:

Programming Language & Frameworks

- Python 3.13.2
- PyTorch for deep learning model implementation
- PyTorch Geometric for graph neural network layers and utilities

Datasets Used

- Cora – Citation Network dataset for node classification
- MUTAG – Molecular Graph dataset for graph classification
- Karate Club – Social network dataset for community detection
- MovieLens (100K) – Used for link prediction with enriched metadata
- Synthetic Datasets – Generated for testing graph generation models

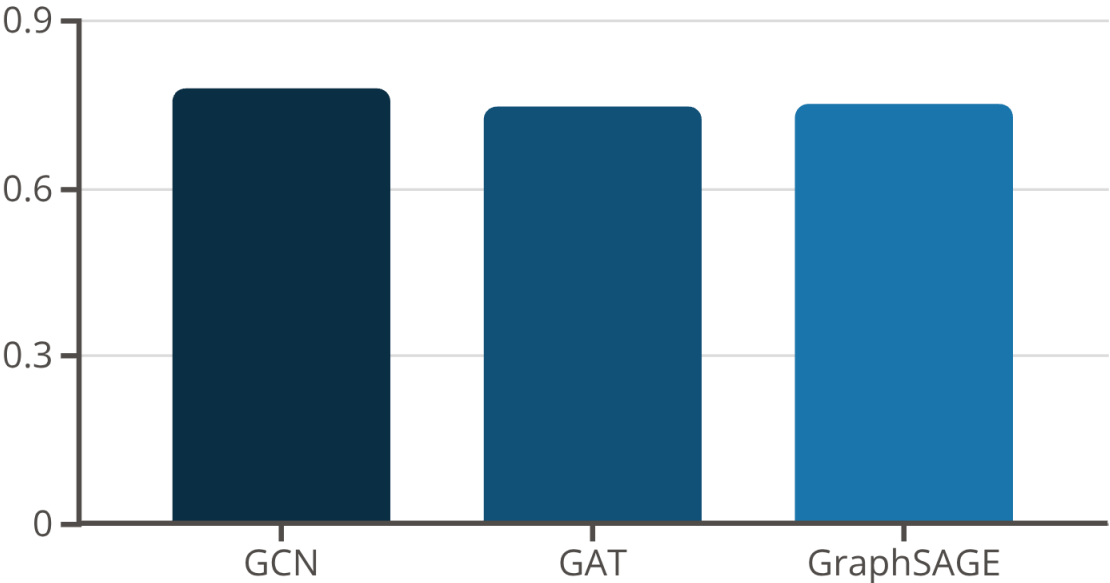
Implemented Models

- Node-level models: GCN, GAT, GraphSAGE
- Graph similarity models: Siamese GCN, GMN
- Generative models: GraphVAE, GraphRNN
- Autoencoders: GAE, GRAE

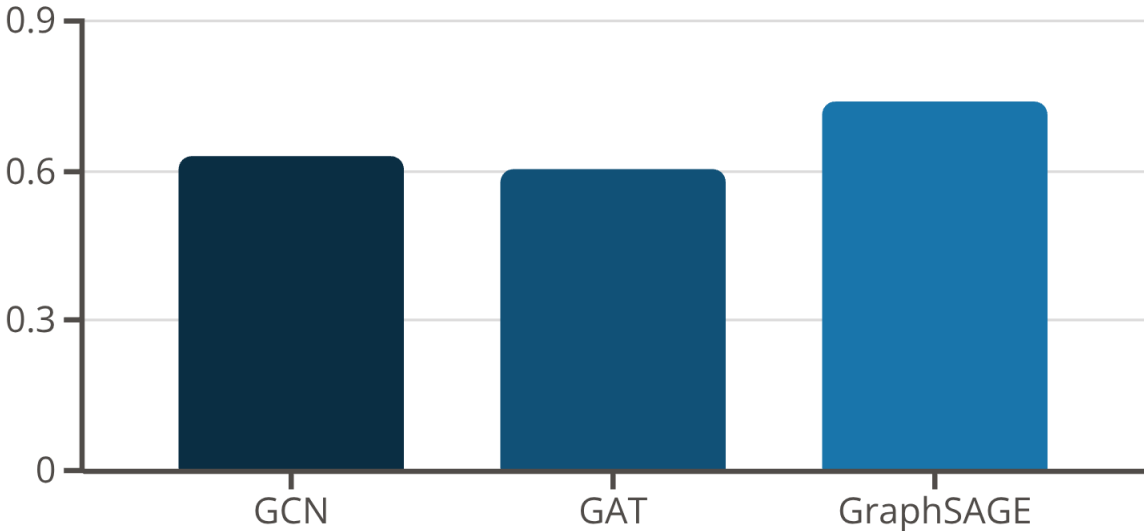
Software Organization

- The implementation is organized in the `codeP` folder, which contains individual Python scripts for each task and model. The `task_runner.py` file handles task execution, including loading datasets, initializing models, and triggering training pipelines.
- The `dataset_loader` scripts handle loading and preprocessing of datasets, converting them into PyG-compatible Data objects. Model scripts define neural network architectures, while training scripts implement optimization, evaluation, and result logging.
- The `codeB` folder contains bash scripts for running the program with a single line command.
- The **modularity of the codebase** ensures that new tasks or models can be integrated by simply adding a new task file and defining the associated configurations, without requiring major architectural changes.

Experimental Results:



Node Classification Performance



Graph Classification Performance

Graph Matching Performance

67%

Siamese GCN

68%

GMN

Graph Reconstruction Performance

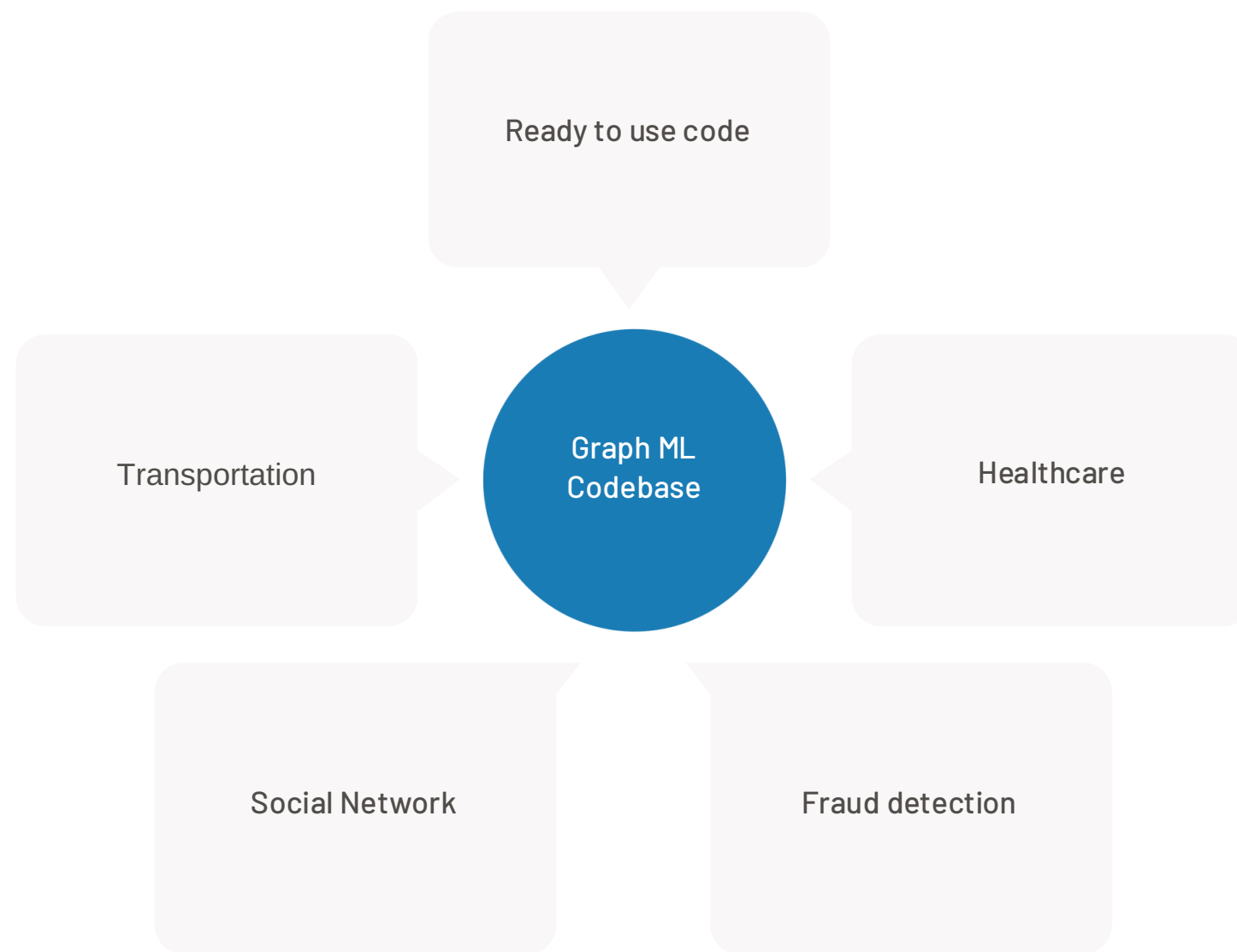
95%

GAE

53%

GRAE

Impact in real life:



Future Improvement:

Moving forward, several promising directions could further enhance the codebase:

- Expanding the codebase to include semi-supervised learning and reinforcement learning methods for graph-based problems.
- Implementing distributed training to handle extremely large-scale graphs more efficiently and reduce training time.
- Expanding the codebase to include other Graph ML tasks.
- Enhancing the system's graph data visualization capabilities to better interpret and explain model predictions.
- Improving the performance of the models for different unfamiliar datasets.



Thank You!